E13 — Binary Tree Level Order Traversal (BFS / Queue)

Experiment: 13 | **LeetCode:** #102 | **CO:** CO2, CO3 | **BT Level:** BT5

Problem Statement

Given the `root` of a binary tree, return the **level order traversal** of its nodes' values (i.e., from left to right, level by level).

Example 1:

```
Input:  root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]
```

```
        3
       / \
      9  20
         / \
        15   7
```

Example 2:

```
Input:  root = [1]
Output: [[1]]
```

Example 3:

```
Input:  root = []
Output: []
```

Constraints:

- The number of nodes is in the range `[0, 2000]`
- `−1000 ≤ Node.val ≤ 1000`

Concept: Breadth-First Search on a Tree

Level order traversal visits nodes level by level, from top to bottom and left to right within each level. This is exactly **Breadth-First Search (BFS)** applied to a tree.

The key technique is to use a **queue** and process all nodes of the current level before moving on:

```
Level 0:        [3]
Level 1:      [9, 20]
Level 2:     [15,  7]
```

At each step, record `len(queue)` before processing — that's the number of nodes at the current level.

Solution 1: BFS with Queue (Standard)

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def levelOrder(root):
    """
    BFS using a queue. Process level by level.
    Time: O(n)  |  Space: O(w)  where w = max width of tree
    """
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)   # Number of nodes at current level
        level_vals = []

        for _ in range(level_size):
            node = queue.popleft()
            level_vals.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level_vals)

    return result
```

Solution 2: Recursive DFS with Level Tracking

Track the current depth during DFS and append values to the corresponding level list.

```
def levelOrder_dfs(root):
    """
    Recursive DFS, passing current depth as parameter.
    Time: O(n)  |  Space: O(h)  where h = height of tree
    """
```

```
    result = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(result):
            result.append([])   # Start a new level
        result[depth].append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)

    dfs(root, 0)
    return result
```

Solution 3: BFS Using Two Lists (No Deque)

```
def levelOrder_two_lists(root):
    """
    Alternating between current and next level lists.
    Time: O(n)  |  Space: O(w)
    """
    if root is None:
        return []

    result = []
    current_level = [root]

    while current_level:
        level_vals = []
        next_level = []

        for node in current_level:
            level_vals.append(node.val)
            if node.left:
                next_level.append(node.left)
            if node.right:
                next_level.append(node.right)

        result.append(level_vals)
        current_level = next_level

    return result
```

Detailed Trace

Tree: `[3, 9, 20, null, null, 15, 7]`

BFS trace (Solution 1):

```
Initial queue: [3]
```

```
Iteration 1:
  level_size = 1
  Pop 3 → level_vals = [3]
    Enqueue 9, 20
  result = [[3]]
  queue = [9, 20]

Iteration 2:
  level_size = 2
  Pop 9  → level_vals = [9]  (no children)
  Pop 20 → level_vals = [9, 20]
    Enqueue 15, 7
  result = [[3], [9, 20]]
  queue = [15, 7]

Iteration 3:
  level_size = 2
  Pop 15 → level_vals = [15] (no children)
  Pop 7  → level_vals = [15, 7] (no children)
  result = [[3], [9, 20], [15, 7]]
  queue = []

return [[3], [9, 20], [15, 7]] ✓
```

DFS trace (Solution 2):

```
dfs(3, depth=0):
  result = [[3]]
  dfs(9, depth=1):
    result = [[3], [9]]
    dfs(None, depth=2) → return
    dfs(None, depth=2) → return
  dfs(20, depth=1):
    result = [[3], [9, 20]]
    dfs(15, depth=2):
      result = [[3], [9, 20], [15]]
      dfs(None, depth=3) → return
      dfs(None, depth=3) → return
    dfs(7, depth=2):
      result = [[3], [9, 20], [15, 7]]
      dfs(None, depth=3) → return
      dfs(None, depth=3) → return

return [[3], [9, 20], [15, 7]] ✓
```

Variations

Zigzag Level Order Traversal (LC #103)

Alternate the direction each level: left-to-right for even levels, right-to-left for odd levels.

```
from collections import deque

def zigzagLevelOrder(root):
    if root is None:
        return []
    result = []
    queue = deque([root])
    left_to_right = True

    while queue:
        level_size = len(queue)
        level = deque()

        for _ in range(level_size):
            node = queue.popleft()
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)    # Reverse insertion
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(list(level))
        left_to_right = not left_to_right

    return result
```

Complexity Summary

Approach	Time	Space
BFS with deque	O(n)	O(w) — max width
Recursive DFS	O(n)	O(h) — tree height
Two-list BFS	O(n)	O(w)

Key Takeaways

- Level order traversal = BFS on a tree; always use a **queue**.
- Record `len(queue)` at the start of each iteration to separate levels.
- The recursive DFS approach is less intuitive but uses less space for tall narrow trees.
- This pattern extends directly to zigzag, bottom-up level order, and right-side view problems.